pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Case Study: FOMC Statement Tone over Time

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Two numbers per statement

Hands-on: read a statement

The course's two word lists and dictionary_score, a count per thousand tokens. Then policy_action, the action a statement states, checked against the rate it states.

Case study: 213 scheduled statements

A dictionary score and a classifier's score for each statement, the classifier fitted only on earlier years, with the label's own words taken out first.

It closes with: what the model shows, and where it fails.

Real documents, described

A statement's count or score

Allowed names	Never called
count of words from the course's list	hawkish or dovish
dictionary score	a signal or a policy view
the action the statement states	a forecast of rates

The classifier's output today

Allowed names	Never called
the classifier's score for raise	a probability, with class weights
the label the model assigns	what the Committee meant
a walk-forward macro F1	a reading of policy

How a dictionary score works

The problem	A number on wording, with no model and nothing hidden
Two lists	Words of firming and words of easing, written down: a choice
The count	Each token on a list is counted every time it appears
The score	(firming minus easing) over tokens, times 1,000, so length does not decide
What comes out	One number per statement that you can check by hand; it reads nothing around a word

The course's word lists

Firming

raise, increase, tighten, firming, elevated

Easing

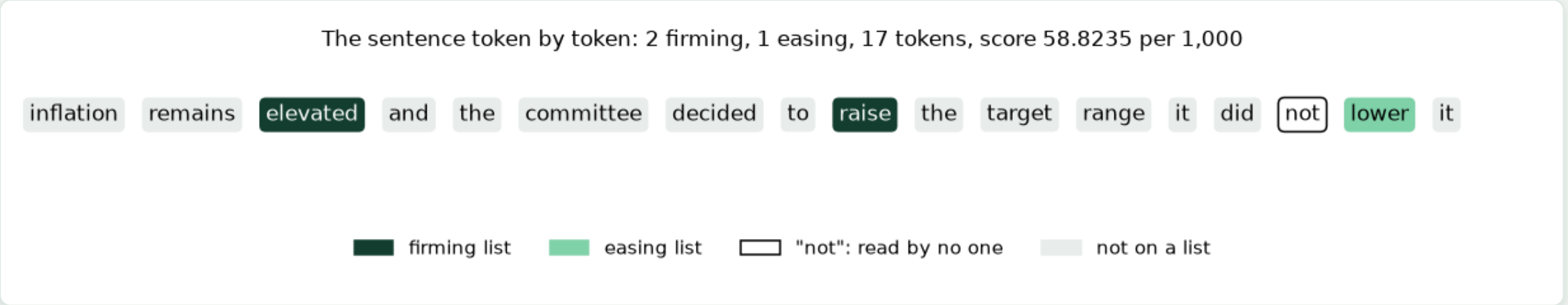
lower, reduce, accommodation, accommodative, easing

```
score = (firming - easing) / tokens * 1000     # points per thousand tokens
"... did not lower it."  ->  lower still counts: 58.8235
```

Named and not used: VADER (its authors: attuned to social media) and Loughran-McDonald (its page: free for use in academic research).

The lists are a choice, one of many: printed in full.

A score, token by token

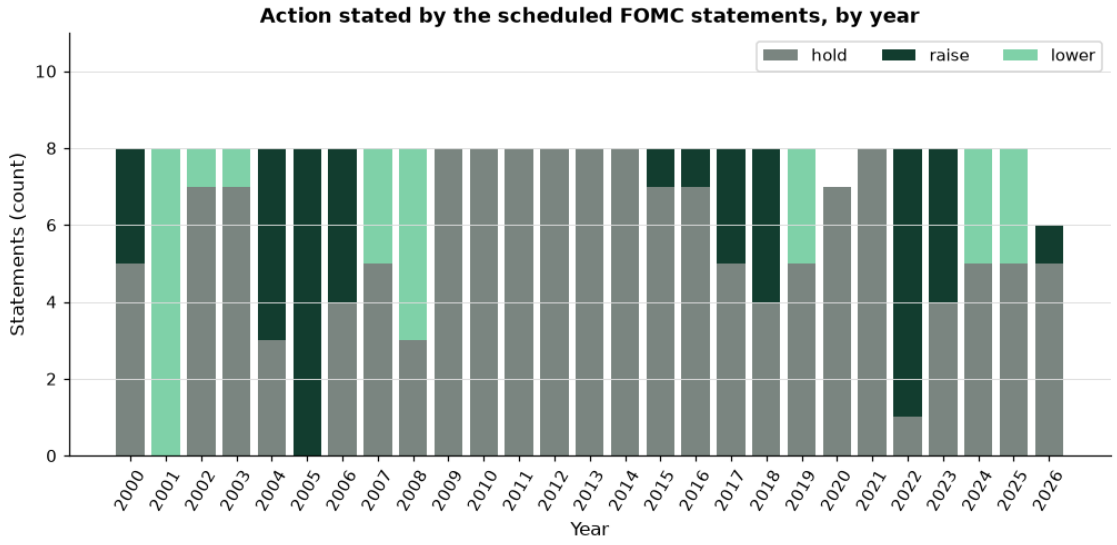


The outlined not counts for nothing: did not lower still adds an easing word.

How the action label is read

The problem	A label from the statement's own words, by a rule and not a model
The sentence	The first that names the federal funds rate, with an action verb before target
Skipped	A sentence about a move at a later meeting
You control	The patterns and the list of later-move phrases
What comes out	raise, lower, or hold per scheduled statement, checked against the stated rate

The action each statement states

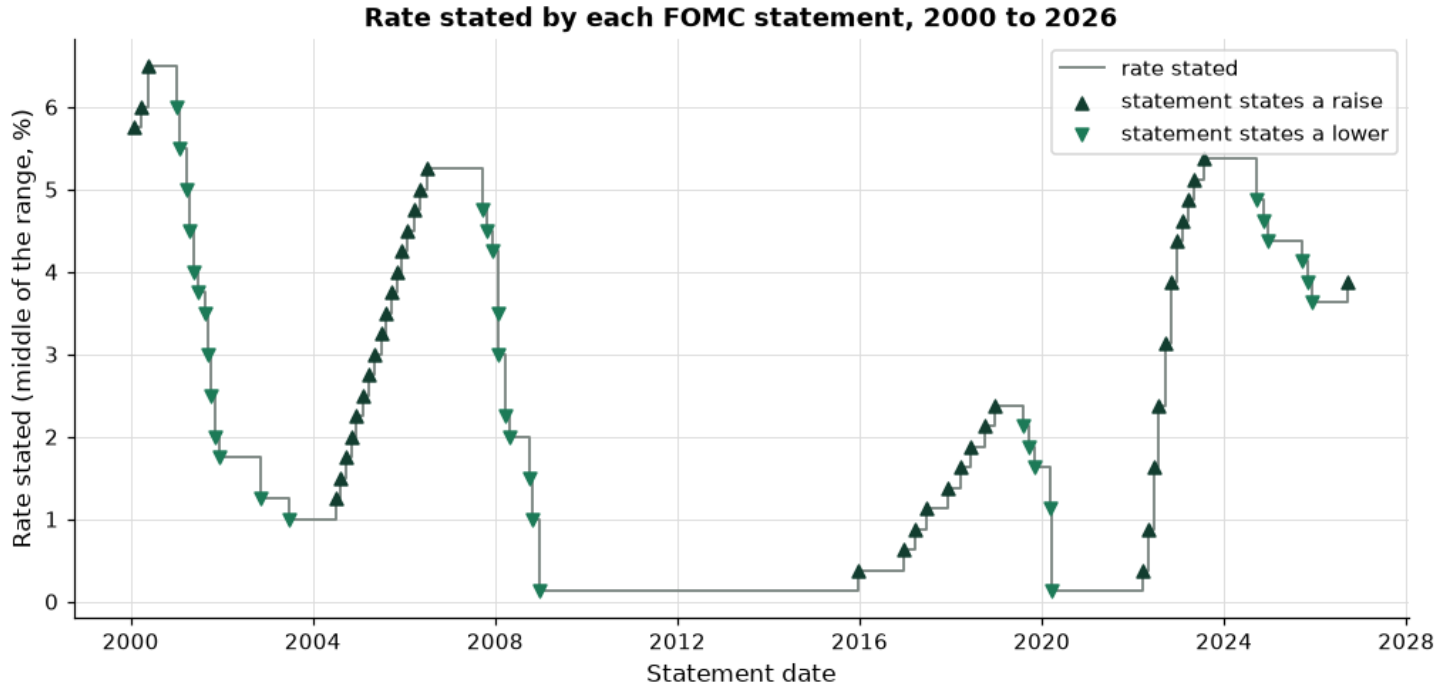


145	hold
41	raise, by policy_action
27	lower
219	checked against the stated rate
219	agree
0	disagree

March 2015: 2 sentences about a raise at a later meeting are skipped ("unlikely", "when it has seen"); the sentence read says hold.

A label is a claim; the stated rate tests it. Case study label: 41 raise, 172 other.

The rate each statement states



What the markers are

Up triangles: the 41 statements that state a raise. Down triangles: the statements that state a lower target, scheduled or not. Each sits at the rate its own statement states, on a step of the line.

Statement dates and stated rates: the test of every label.

The case study

How did a count of the course's words move from 2000 to 2026, and can a classifier fitted only on earlier statements tell, from the rest of the text, which announced a raise?

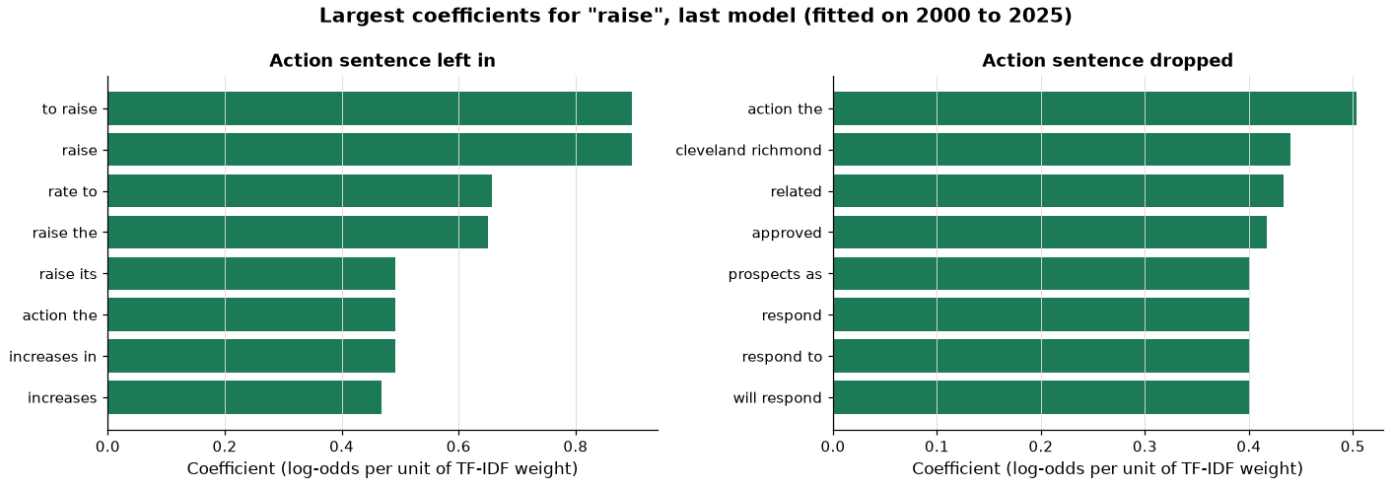
1. Take the label's own words out first, after showing the leak
2. A dictionary score for every statement
3. The classifier, walk-forward by year from 2005: 173 scored
4. Side by side, with a bootstrap interval, and a workbook
5. What the model shows, and where it fails

40 statements of 2000 to 2004 are fitted on only. 173 statements in 22 years are scored, 33 of them raises.

How the walk-forward, the sentence drop, and the interval work

Walk-forward	Per year: fit a fresh model on earlier years only, score that year; measure all years once
Past only	assert_past_only stops a fold whose training dates are not all earlier
drop_sentences	Deletes every sentence matching a pattern and keeps the rest in order
Bootstrap	Resample the scored statements 1,000 times; 2.5th and 97.5th percentiles are the interval
You control	The first year, the window, the pattern, the number of resamples

The label is in the text



Macro F1, leak in: 0.6562

The label's own words top the coefficients, yet the score is not near 1.0. A leak does not need to lift the score to be a leak.

467

sentences dropped

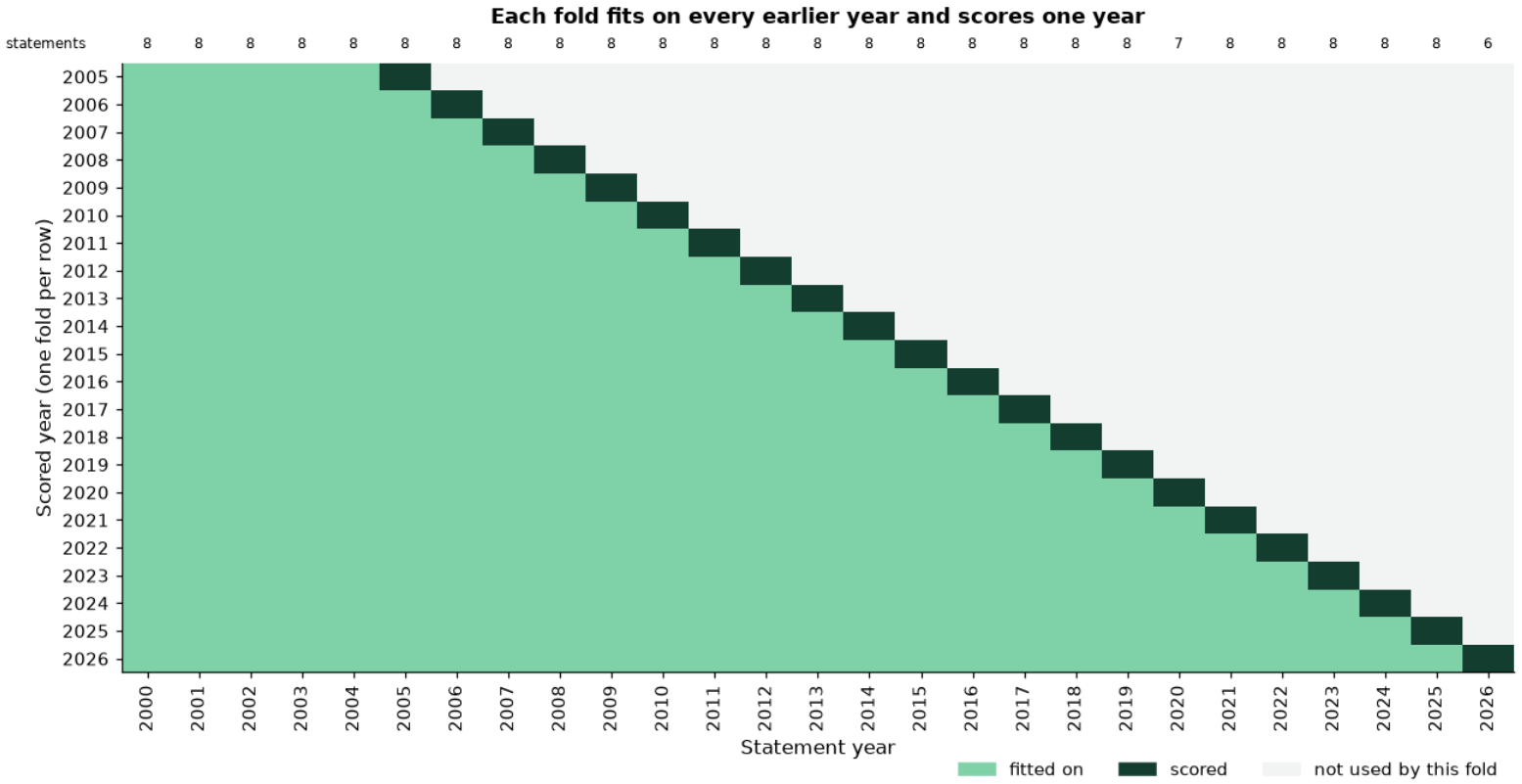
0

kept texts that still state an action

0.8046

share of tokens kept

The folds of a walk-forward



22

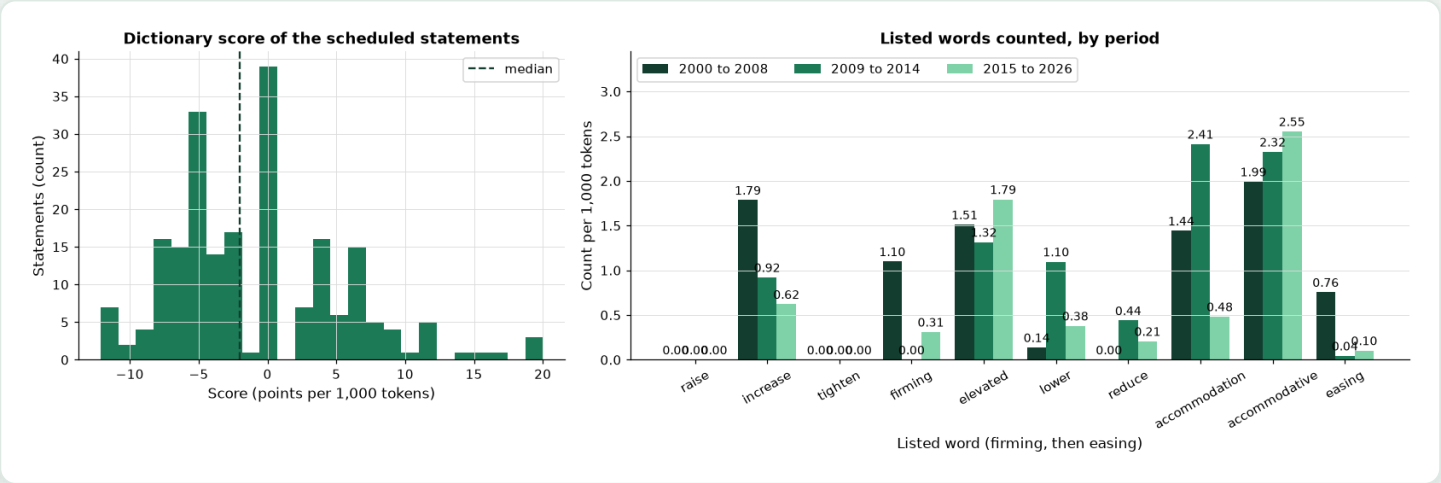
folds, one a year

40

first fold fits on

The last fold fits on 207. The fitting set only grows; no fold ever touches a later year.

Two numbers per statement





Model	Macro F1	Accuracy	Recall
majority_label	0.4473	0.8092	0.0000
tfidf_logistic_balanced_action_left_in	0.6562	0.7688	0.5152
tfidf_logistic_balanced	0.6510	0.7630	0.5152

Walk-forward on the kept text: 0.6510 against 0.4473 for the majority label.

Raise and tighten are gone with the rate sentences; elevated and accommodative carry the score.

A second, smaller leak

Term (kept text)	Coefficient
action the	0.5036
cleveland richmond	0.4404
related	0.4338
approved	0.4172

In a related action…

…the Board of Governors approved an increase in the discount rate: beside a change in the target, and it names no federal funds rate.

20

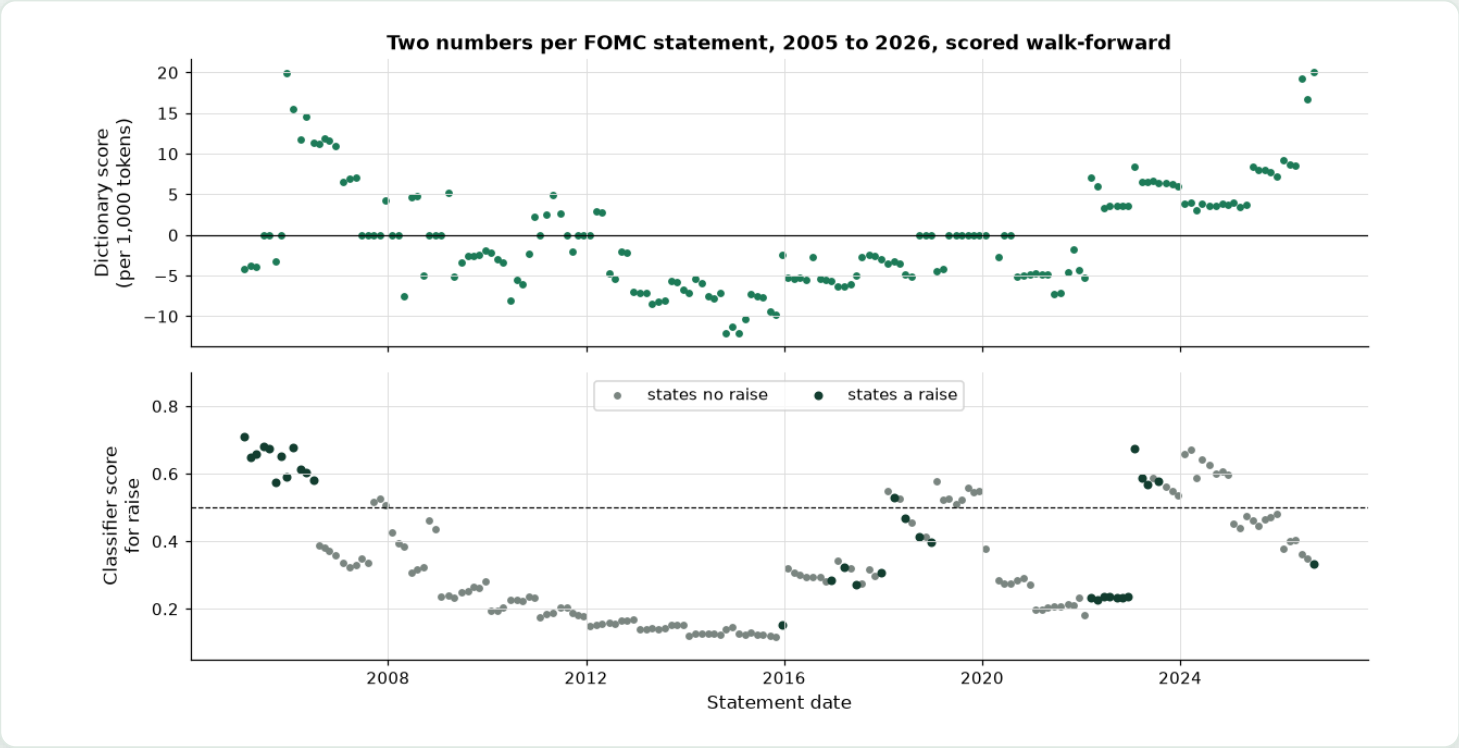
raise statements that hold it

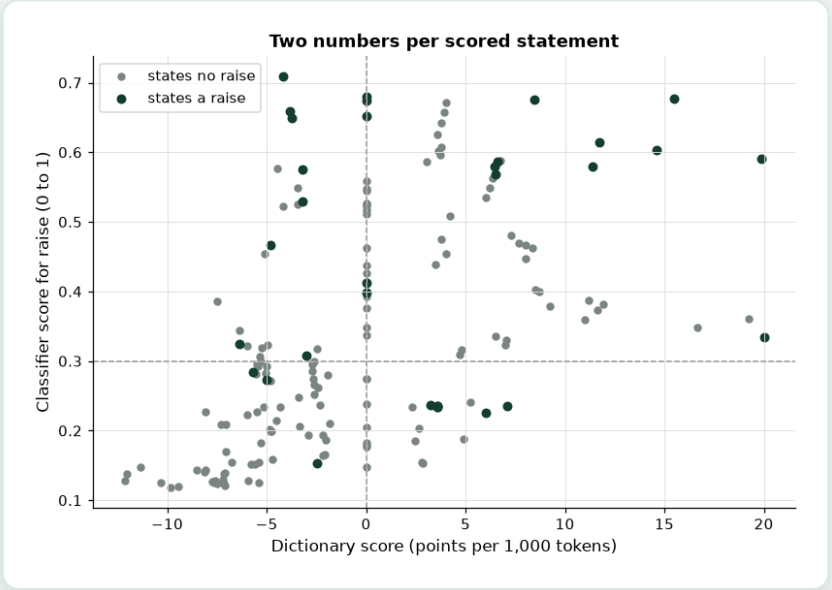
18

other statements, mostly cuts

No pattern written for the first leak removes this one.

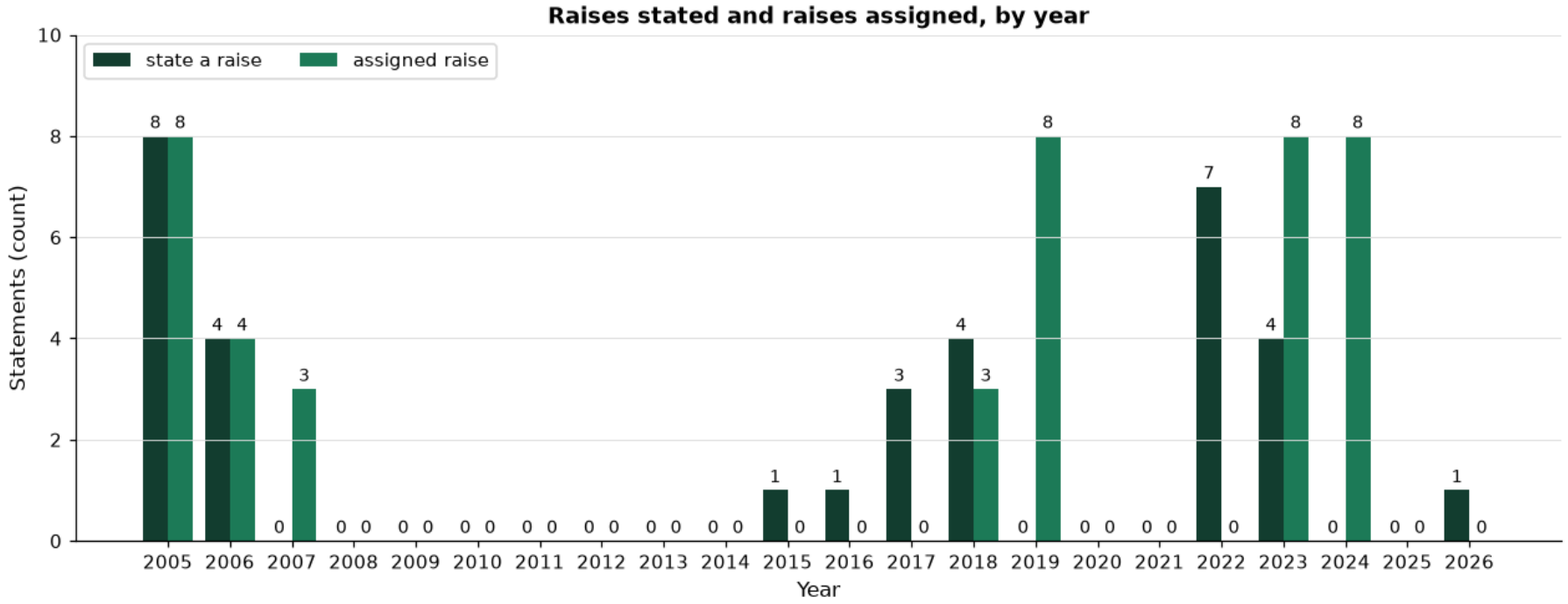
Side by side, by date





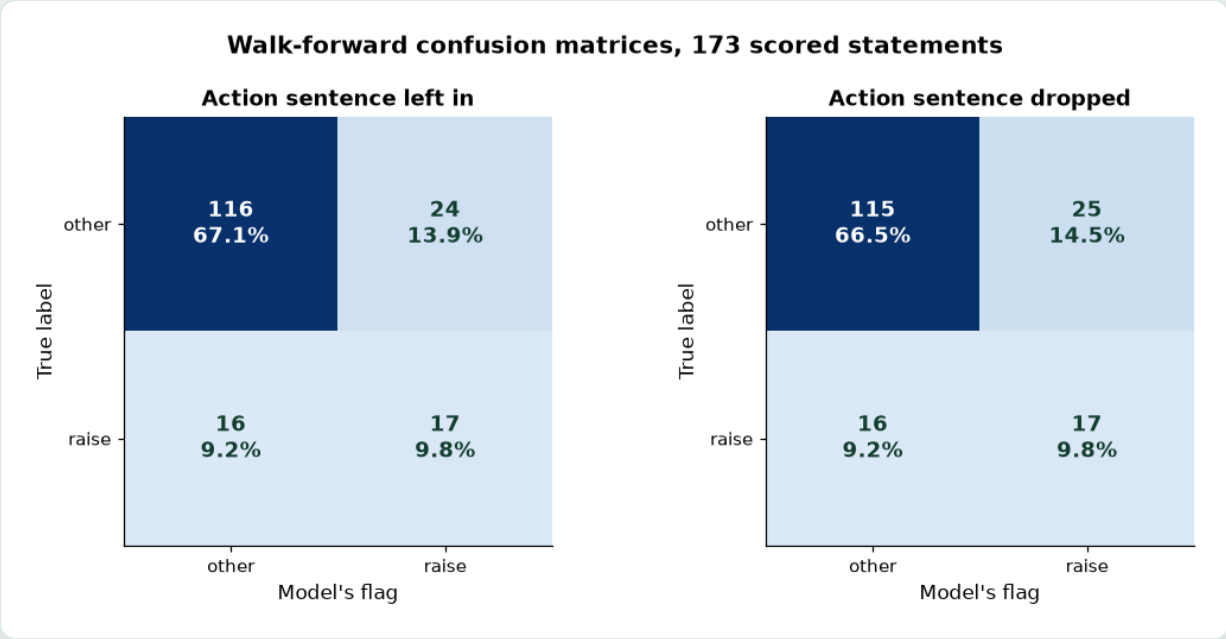
Same side of the medians: 0.6821 (both above 45, both at or below 73); rank correlation 0.5898. Scores move in year blocks.

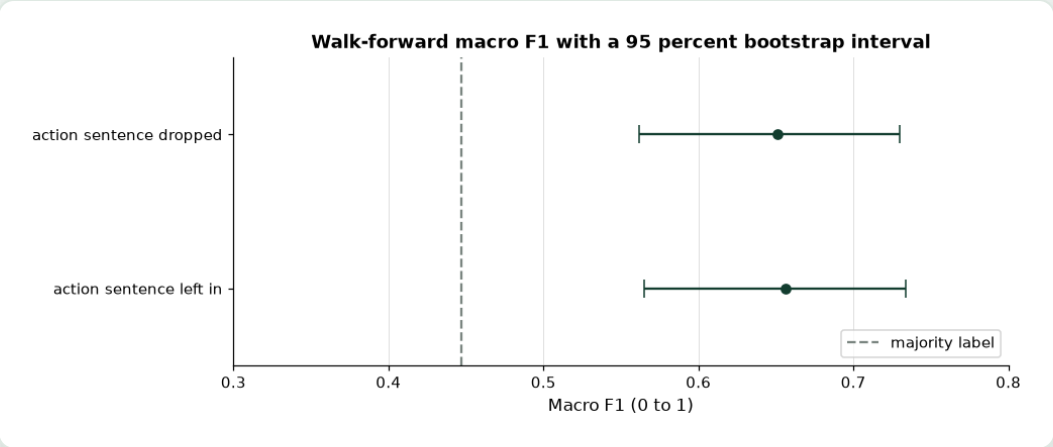
Year by year



19 of 22 scored years get one label for every statement; raise throughout: 2005, 2019, 2023, 2024. A year comes out as a block.

Two runs, side by side





Dropped: macro F1 0.6510, interval 0.5615 to 0.7297, against 0.4473 for the majority label. Every statement was scored by a model fitted on earlier years only: there is no training or test panel.

What the model shows, and where it fails

0.6510

macro F1, interval 0.5615 to 0.7297

19 of 22

years labelled as one block

16 of 33

raises missed; 25 false flags

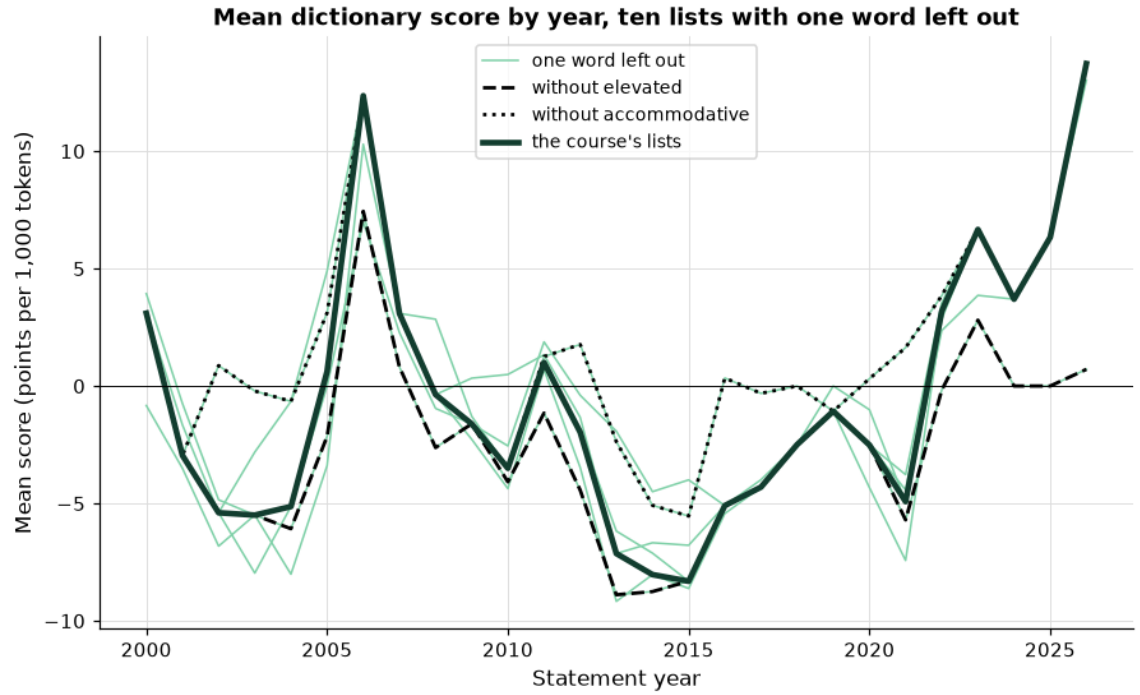
The label	patterns fit 2000 to 2026; a new wording needs the stated-rate check again
The leak	the related action on the discount rate is still in the text
The years	whole years one way: raise throughout 2005, 2019, 2023, 2024
The words	not lower still counts; raised and increases do not
The interval	statements of one year are alike: narrower than it should be

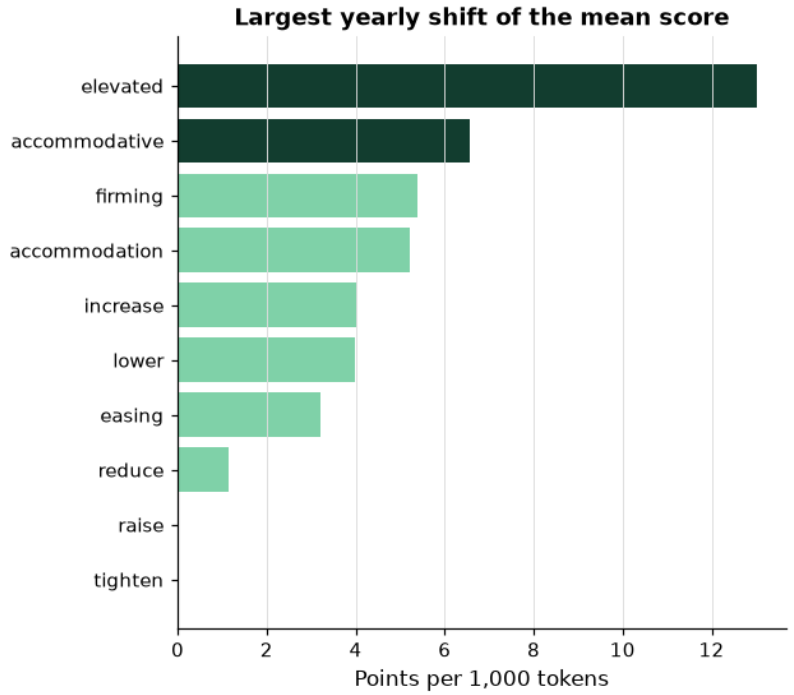
Both numbers describe the wording of statements already published. Source: FOMC post-meeting statements, Board of Governors of the Federal Reserve System, federalreserve.gov.

How a leave-one-out test and a training window work

The problem	A choice made once can be tested by changing it alone
Leave one out	Drop one word from the lists, recompute the yearly mean score, see how far it moves
The window	Let each fold use only the last few years, and score the same statements
You control	Which choice to vary, and by how much
What comes out	How far a result moves when one choice changes: set it beside the bootstrap interval

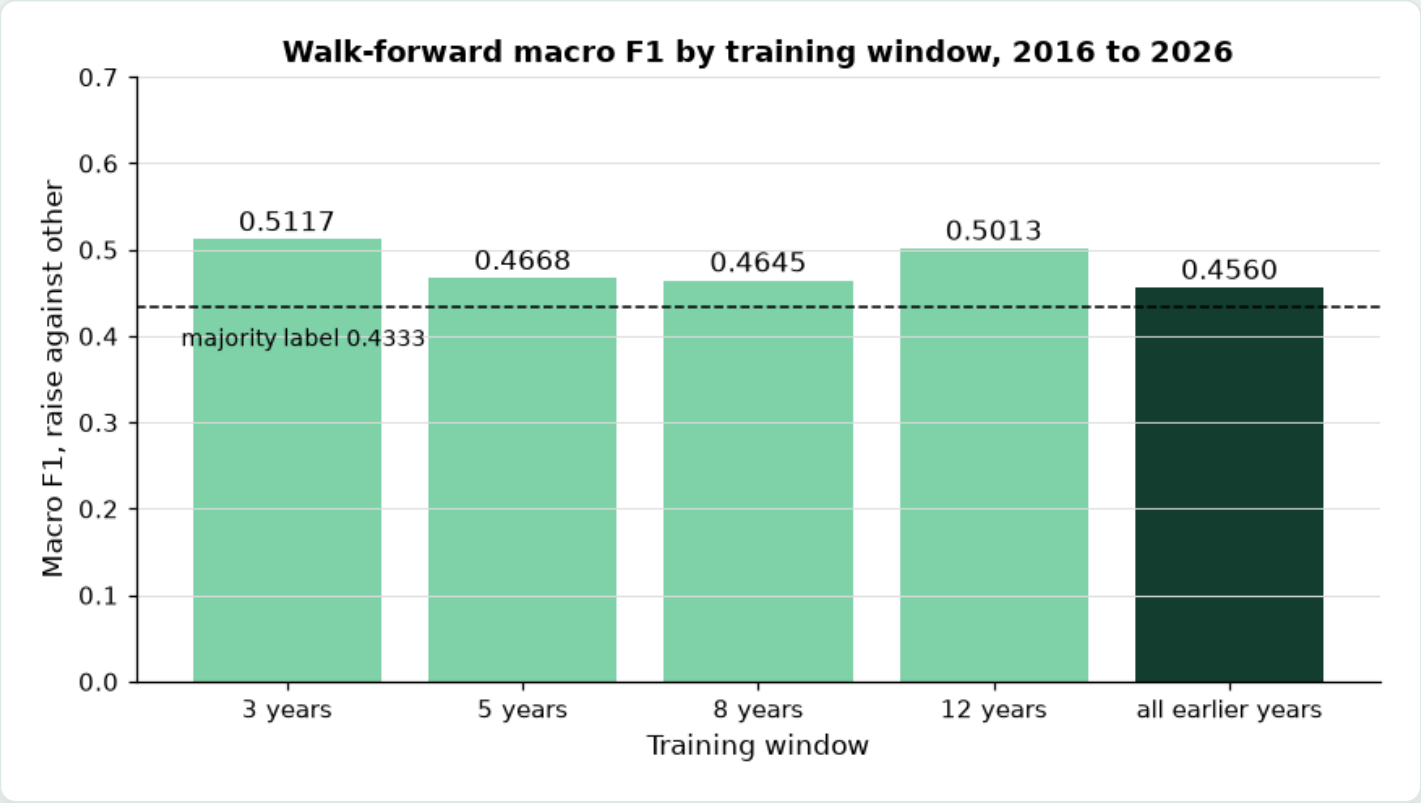
Ten lists, one word left out





One word, elevated, carries most of the score. Raise and tighten move nothing: the drop of section 5.4 removed them from the text.

Five training windows



No window is shown to be better: the spread of macro F1 is smaller than the 95 percent interval of the case study's run.

A window of 3 years holds no raise in one year, so the function gives that year the one label it has seen, and counts it.

Every number describes statements already published; no other rows exist to hold back.

Excel side by side

Excel side by side (tested in Excel)

`=SUMPRODUCT((LEN(LOWER(A2))-LEN(SUBSTITUTE(LOWER(A2),pos,"")))/LEN(pos))`	2 on 2008-09-16; whole words 0
`=SUMPRODUCT((LEN(LOWER(A2))-LEN(SUBSTITUTE(LOWER(A2),neg,"")))/LEN(neg))`	1; whole words 1
`=(C2-D2)/B2*1000`	4.9505; whole words -4.9505
`pos = H1:H5, neg = I1:I5 (named ranges)`	equal to str.count on all 213 rows

97

statements where the counts differ

268 / 194

firming: substrings / whole words

356 / 315

easing: substrings / whole words

"increasing" holds "easing"; "slower" holds "lower". No tokenizer in Excel.

Save the day: tag week 1

1. Commit

the modules and today's textkit

2. Log

results.csv, four decimals

3. Tag

git tag week1

4. Diff

week1~1 against week1

```
week1
results.csv | 4 ++++
1 file changed, 4 insertions(+)
```

pytest: 31 passed. A tag names one commit for good.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: "Not" with a capital N is never a token: 0 of 213 scores change, though 26 texts hold "not".

Code that runs is not the same as code that is right.

What you can do now

- Write a word list, print it, and score per thousand tokens
- Test a label read from text against another reading
- Find the label in the text and take it out first
- Score dated documents walk-forward by year
- Count list words in Excel, and know what a substring is

Week 2, Day 1 Sentence embeddings and semantic search